

Should concepts be enabled or disabled?

Document #: P1871R0
Date: 2019-10-06
Project: Programming Language C++
LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Motivation

The `sized_range` concept is currently defined as follows, in [\[range.sized\]](#):

```
template<class T>
concept sized_range =
    range<T> &&
    !disable_sized_range<remove_cvref_t<T>> &&
    requires(T& t) { ranges::size(t); };
```

The reason for the extra

`!disable_sized_range<remove_cvref_t<T>>` check is that some types can meet the syntactic requirements of `ranges::size` without meeting the semantic requirement that this call must have constant time. For instance, a pre-C++11 `std::list` had `O(N)` `size()`, but this wouldn't be detectable, so the type trait exists to allow for such containers to opt out of being considered `sized_ranges`.

A similar observation can be made for the `sized_sentinel_for` concept, from [\[iterator.concept.sizedsentinel\]](#):

```
template<class S, class I>
concept sized_sentinel_for =
    sentinel_for<S, I> &&
    !disable_sized_sentinel_for<remove_cv_t<S>, remove_cv_t<I>> &&
    requires(const I& i, const S& s) {
        { s - i } -> same_as<iter_difference_t<I>>;
        { i - s } -> same_as<iter_difference_t<I>>;
    };
```

On the flip side, we also have the `view` concept in [\[range.view\]](#):

```
template<class T>
concept view =
    range<T> && semiregular<T> && enable_view<T>;
```

Two negated type traits for disabling, one positive one to enable. Why both directions?

§ 1.1 To enable or not to disable, that is the question

One argument that you could make comes from usage. Pretty much all types that meet the syntactic requirements for `sized_range` and `sized_sentinel` do in fact model those concepts. It's only a few oddballs that need to explicitly *opt-out* of those concepts. On the other hand, most types that meet the syntactic requirements for `view` aren't actually views - and those need to *opt-in*.

A different argument can be: what does it mean to specialize these traits? If you specialize `disable_sized_sentinel` to be `true`, that type is *definitely* not a `sized_sentinel`. But if you specialize `enable_view` to be `true`, that type isn't necessarily a view - it's just that you checked that one box. It's not as final a statement.

But ultimately, double negatives are needlessly difficult to understand. We say a type models `sized_range` or it does not model `sized_range`. We do not say a type does not not model `sized_range`. We really should try to avoid double negatives whenever possible.

The real problem is that we need this variable template to begin with - but that's a separate, language problem. If we rename `disable_sized_range` to `enable_sized_range` and `disable_sized_sentinel` to `enable_sized_sentinel_for`, then all of our type trait variable templates are spelled the same way (`enable_concept_name`).

The traits will have different defaults, but are defined such that they should rarely need to be touched anyway (`enable_view` has a heuristic trying to be right most of the time, and the other two in this new formulation would almost always be `true`).

§ 2 Proposal

The proposal is to flip the two disabling variable templates to enabling variable templates: renaming `disable_sized_range` to `enable_sized_range` and `disable_sized_sentinel` to `enable_sized_sentinel_for` (note the extra `_for`). And then update all usage of them to be positive rather than negative.

Change 23.2 [iterator.synopsis]:

```
// [iterator.concept.sizedsentinel], concept sized_sentinel_for
template<class S, class I>
- inline constexpr bool disable_sized_sentinel = false;
+ inline constexpr bool enable_sized_sentinel_for = true;
```

and later:

```
template<class Iterator1, class Iterator2>
    requires (!sized_sentinel_for<Iterator1, Iterator2>)
- inline constexpr bool
disable_sized_sentinel<reverse_iterator<Iterator1>,
-
reverse_iterator<Iterator2>> = true;
+ inline constexpr bool
enable_sized_sentinel_for<reverse_iterator<Iterator1>,
+
reverse_iterator<Iterator2>> = false;
```

Change 23.3.4.8 [iterator.concept.sizedsentinel]:

- 1 The `sized_sentinel_for` concept specifies requirements on an `input_or_output_iterator` and a corresponding `sentinel_for` that allow the use of the `-` operator to compute the distance between them in constant time.

```
template<class S, class I>
concept sized_sentinel_for =
    sentinel_for<S, I> &&
-   !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>>> &&
+   enable_sized_sentinel_for<remove_cv_t<S>, remove_cv_t<I>>> &&
    requires(const I& i, const S& s) {
        { s - i } -> same_as<iter_difference_t<I>>;
        { i - s } -> same_as<iter_difference_t<I>>;
    };
```

- 2 Let `i` be an iterator of type `I`, and `s` a sentinel of type `S` such that `[i, s)` denotes a range. Let `N` be the smallest number of applications of `++i` necessary to make `bool (i == s) be true`. `S` and `I` model `sized_sentinel_for<S, I>` only if

```
- [2.1]{.pnum} If `N` is representable by `iter_difference_t<I>`,
then `s - i` is well-defined and equals `N`.
- [2.2]{.pnum} If `-N` is representable by `iter_difference_t<I>`,
then `i - s` is well-defined and equals `-N`.
```

```
template<class S, class I>
-   inline constexpr bool disable_sized_sentinel = false;
+   inline constexpr bool enable_sized_sentinel_for = true;
```

- 3 *Remarks:* Pursuant to [namespace.std], users may specialize `disable_sized_sentinel` `enable_sized_sentinel_for` for cv-unqualified non-array object types `S` and `I` if `S` and/or `I` is a program-defined type. Such specializations shall be usable in constant expressions ([`expr.const`]) and have type `const bool`.
- 4 [*Note:* `disable_sized_sentinel` `enable_sized_sentinel_for` allows use of sentinels and iterators with the library that satisfy but do not in fact model `sized_sentinel_for`. — *end note*]

Change 24.2 [ranges.syn]:

```
// [range.sized], sized ranges
template<class>
-   inline constexpr bool disable_sized_range = false;
+   inline constexpr bool enable_sized_range = true;
```

Change 24.3.9 [range.prim.size]:

- 1 The name `size` denotes a customization point object. The expression `ranges::size(E)` for some subexpression `E` with type `T` is expression-equivalent to:

- (1.1) — decay -copy (extent_v <T >) if T is an array type ([basic.compound]).
- (1.2) — Otherwise, if `disable_sized_range` <remove_cv_t <T >> `enable_sized_range` <remove_cv_t <T >> ([range.sized]) is `false` `true`:
 - (1.2.1) — decay -copy (E .size ()) if it is a valid expression and its type T is integer-like ([iterator.concept.winc]).
 - (1.2.2) — Otherwise, decay -copy (size (E)) if it is a valid expression and its type T is integer-like with overload resolution performed in a context that includes the declaration: [...]

Change 24.4.3 [range.sized]:

- 1 The sized_range concept specifies the requirements of a range type that knows its size in constant time with the size function.

```
template<class T>
concept sized_range =
    range<T> &&
-    !disable_sized_range<remove_cvref_t<T>> &&
+    enable_sized_range<remove_cvref_t<T>> &&
    requires(T& t) { ranges::size(t); };
```

and

```
template<class>
-    inline constexpr bool disable_sized_range = false;
+    inline constexpr bool enable_sized_range = true;
```

- 4 *Remarks:* Pursuant to [namespace.std], users may specialize `disable_sized_range` `enable_sized_range` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions ([expr.const]) and have type `const bool`.
- 5 [*Note:* `disable_sized_range` `enable_sized_range` allows use of range types with the library that satisfy but do not in fact model sized_range. — end note]